Experiment 9

Student Name: Umang Kaushik **UID: 23BCS10712**
Branch: BE CSE **Section/Group: KRG-1-B**
Semester: 6th **Date of Performance:18th Apr**
Subject Name: SD **Subject Code: 23CSH-314**

1. Functional Requirements (FRs)

- Search events by title, city, date
- View event details (metadata + seat availability)
- View seat map and real-time availability
- Lock seats before booking (temporary reservation)
- Complete ticket booking with payment
- Receive booking confirmation via notification

2. Non-Functional Requirements (NFRs)

- Scale: 100M DAU via CDN, load balancing, caching
- Availability (AP – Search path):
  - Search & event viewing must not fail
  - Slightly stale data acceptable (cache + Elasticsearch)
- Consistency (CP – Booking path):
  - Strong consistency for seat booking
  - No double booking (locks + ACID DB)
- Low Latency:
  - Cached reads (<200ms), booking (<1–2s)
- Fault Tolerance:
  - Async processing via MQ
- Security:
  - Payment safety + auth

3. API Design

Search (via Search Service → Elasticsearch)
GET /search?title=&city=&date=
Event Details (via Event Catalog Service + Cache)
GET /events/{event_id}
Seat Availability (Redis Cache first)
GET /events/{event_id}/seats
Lock Seats (Redis / Booking Service)
POST /seats/lock
Body: {user_id, show_id, seat_ids}
Book Ticket (Booking Service → DB)
POST /bookings
Body: {user_id, show_id, seat_ids}
Payment (Payment Service)
POST /payments
Body: {booking_id, amount}

4. Core Entities

- User
- Event
- Show
- Venue
- Seat
- ShowSeat (stateful seat per show)
- Booking
- Payment
- SeatLock
- Notification

5. Database Schema Design

Primary DB (Postgres – Source of Truth, ACID)
ShowSeat (critical for consistency)

show_seat_id (PK)
show_id
seat_id
status (AVAILABLE, LOCKED, BOOKED)
version
Booking
booking_id (PK)
user_id
show_id
status
created_at
Booking_Seat
id
booking_id
show_seat_id
Payment
payment_id
booking_id
status
amount

6. High-Level Design (HLD)

1. Entry Layer

- CDN + WAF
  - Static caching, security
- API Gateway / Load Balancer
  - Auth, routing, rate limiting

2. Service Layer

Search Service (AP)

- Queries Elasticsearch

- Always available
- Uses async index updates

Event Catalog Service (AP)

- Reads from:
  - Redis (cache hit)
  - Read replicas (fallback)

Booking Service (CP)

- Handles critical booking flow
- Ensures strong consistency via:
  - DB row locking OR
  - Redis distributed locks

Payment Service

- Executes secure transactions
- Called synchronously from Booking Service

3. Data Layer

Primary DB (Postgres)

- ACID guarantees
- Handles booking writes

Read Replicas

- Async replication
- Serve read-heavy traffic

Redis Cache

- Fast access for:
  - Event info
  - Seat availability
- Cache invalidation after booking

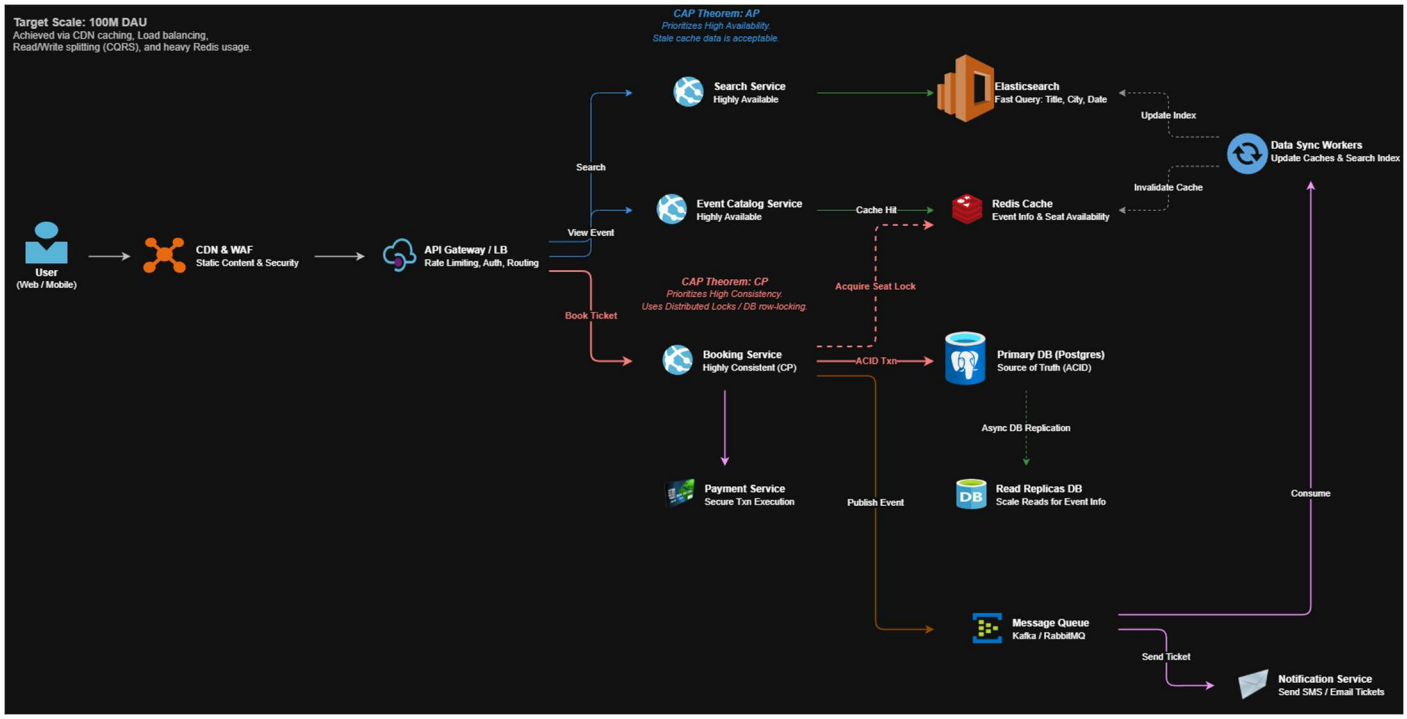
Elasticsearch

- Optimized search queries
- Updated asynchronously

4. Messaging Layer

Message Queue (Kafka / RabbitMQ)

- Booking events published
- Used for:
  - Notifications
  - Cache invalidation
  - Search index updates

Data Sync Workers

- Consume MQ events
- Update:
  - Redis cache
  - Elasticsearch index